

### **Introduction to System:**

The word System is derived from Greek word Systema, which means an organized relationship between any set of components to achieve some common cause or objective

A system is “an orderly grouping of interdependent components linked together according to a plan to achieve a specific goal.”

System is a group of elements or component which work together to accomplish a common task

### **Properties of a System**

#### **Organization**

Organization implies structure and order. It is the arrangement of components that helps to achieve predetermined objectives

#### **Interaction**

It is defined by the manner in which the components operate with each other.

For example, in an organization, purchasing department must interact with production department and payroll with personnel department.

#### **Interdependence**

Interdependence means how the components of a system depend on one another. For proper functioning, the components are coordinated and linked together according to a specified plan. The output of one subsystem is the required by other subsystem as input.

#### **Integration**

Integration is concerned with how a system components are connected together. It means that the parts of the system work together within the system even if each part performs a unique function.

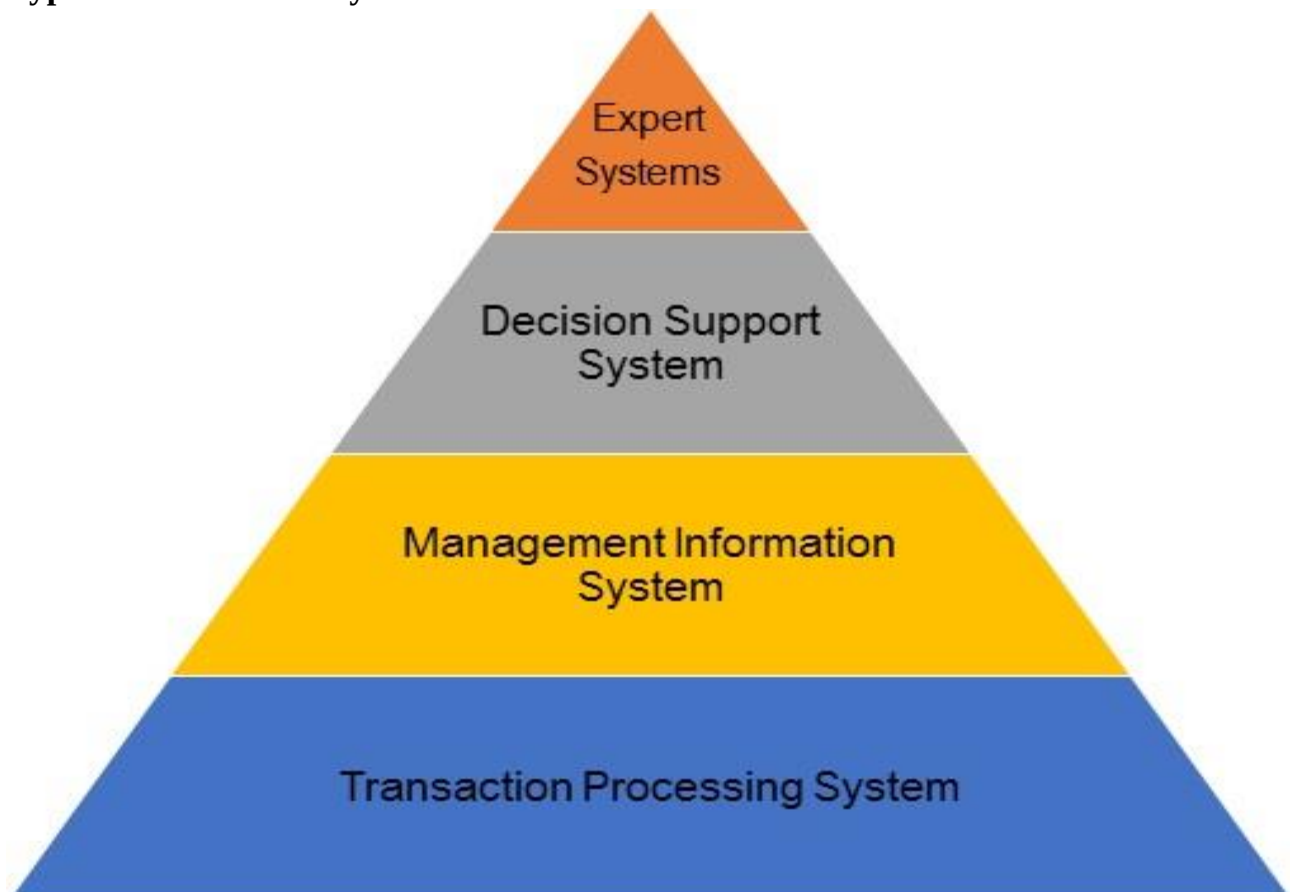
## **Information System and its types:**

Information system is group of people, information technology, and business process to achieve a business goals. Information systems are a group of interconnected elements working to collect, process, store, and distribute data to provide coordination, visualization in an organization, analysis, and decision-making.

An information system (IS) is an interconnected set of components used to collect, store, process and transmit data and digital information. At its core, it is a collection of hardware, software, data, people and processes that work together to transform raw data into useful information. An IS supports a variety of business objectives such as improved customer service or increased efficiency

The Information system can be represented as a set of software, hardware, and telecommunications network that people create and use to gather, produce, and distribute beneficial data, generally in organizational settings.

### **Types of Information System**



## **Experts Systems**

Experts Systems include knowledge to assist management in identifying and fixing problems. These systems are based on artificial intelligence research concepts

An expert system's components include a knowledgebase and software modules. These modules make inferences based on knowledge and respond to a user's query

## **Decision Support System (DSS)**

- The end-user is more active in producing DSS than a MIS since it includes tools and strategies to assist in obtaining relevant information and analyzing possibilities and alternatives.
- Examples include Financial Planning Systems and Bank Loan Management Systems.

## **Transaction Processing System (TPS)**

- The term "transaction processing system" refers to an information system that processes data arising from business transactions
- Batch processing and online transaction processing are the two methods used to complete the transaction.
- Examples include a billing system, a payroll system, and a stock control system, etc.

## **Management Information System (MIS)**

- The purpose of a Management Information System is to turn relatively raw data available through a Transaction Processing System into a summarised and aggregated form for the manager, generally in the form of a report. Middle management and operational supervisors are likely to use the reports.
- Examples include Sales Management Systems and Human Resource Management Systems.

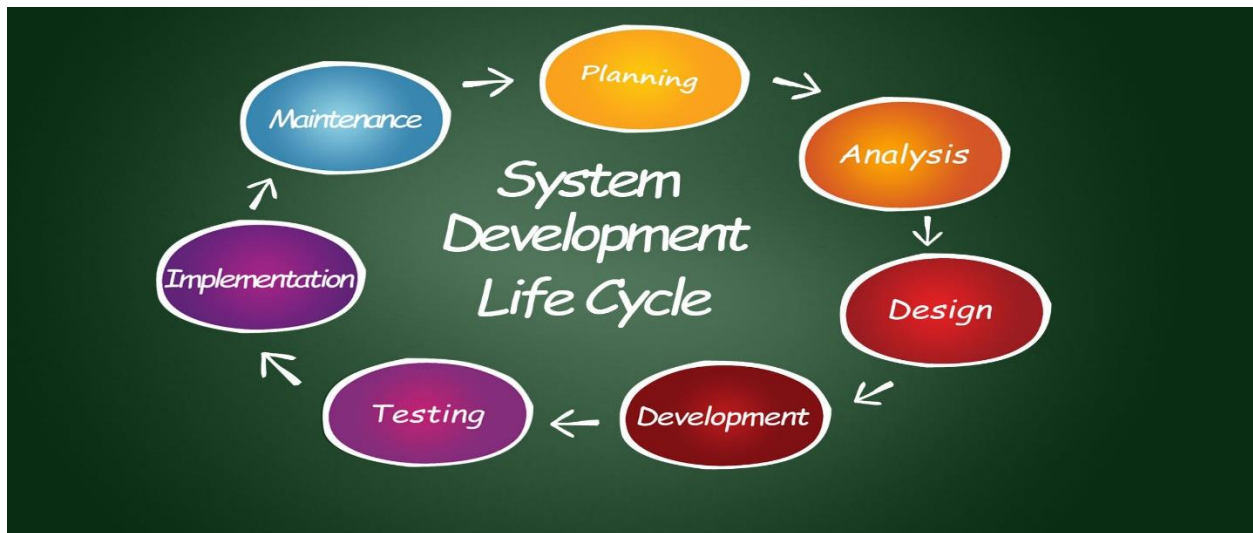
Data management involves maintaining the accuracy, consistency and integrity of the data stored in the system

Network management involves monitoring and maintaining the network infrastructure to keep it functioning correctly

Security is critical for an IS because it is vulnerable to threats such as hacking, viruses, malware and unauthorized access.

# System Development Life Cycle

The **System Development Life Cycle (SDLC)** provides a well-structured framework that gives an idea, of how to build a system. It consists of steps as follows – Plan, Analyze, Design, Develop, Implement and Maintain. In this article, we will see all the stages of system development.



## 1.Planning:

The Planning stage lays the groundwork for the entire SDLC. It involves identifying the system's goals, defining project scope, setting objectives, establishing timelines, and determining available resources. Planning ensures that the development process aligns with organizational needs and sets a clear direction for subsequent stages.

## 2: Analyze:

During the Analysis stage, the focus is on gathering and understanding the requirements of the system. This includes conducting interviews, studying existing processes, and identifying stakeholders' needs. The gathered information serves as a basis for designing a system that meets users' expectations and addresses organizational challenges

## 3: Design:

System Design is a critical stage in the SDLC, where the requirements gathered during the Analysis phase are translated into a detailed technical plan. It involves designing the system's architecture, database structure, and

user interface, and defining system components. The Design stage lays the foundation for the subsequent development and implementation phases

#### **4: Develop/Implementation:**

The Development stage involves the actual coding and programming of the system. Based on the design specifications, developers write code, create database structures, and implement necessary functionalities. Rigorous testing and quality assurance are performed to ensure the system's accuracy, performance, and adherence to the design requirements

#### **5. Testing:**

Testing is a crucial phase where the system is evaluated to ensure that it meets the specified requirements and functions correctly. This includes various types of testing such as unit testing, integration testing, system testing, and user acceptance testing

**6. Deployment:** Once the system has been thoroughly tested and approved by stakeholders, it is deployed to the production environment. This may involve installation, configuration, and data migration activities to ensure a smooth transition to the new system.

#### **7: Maintenance:**

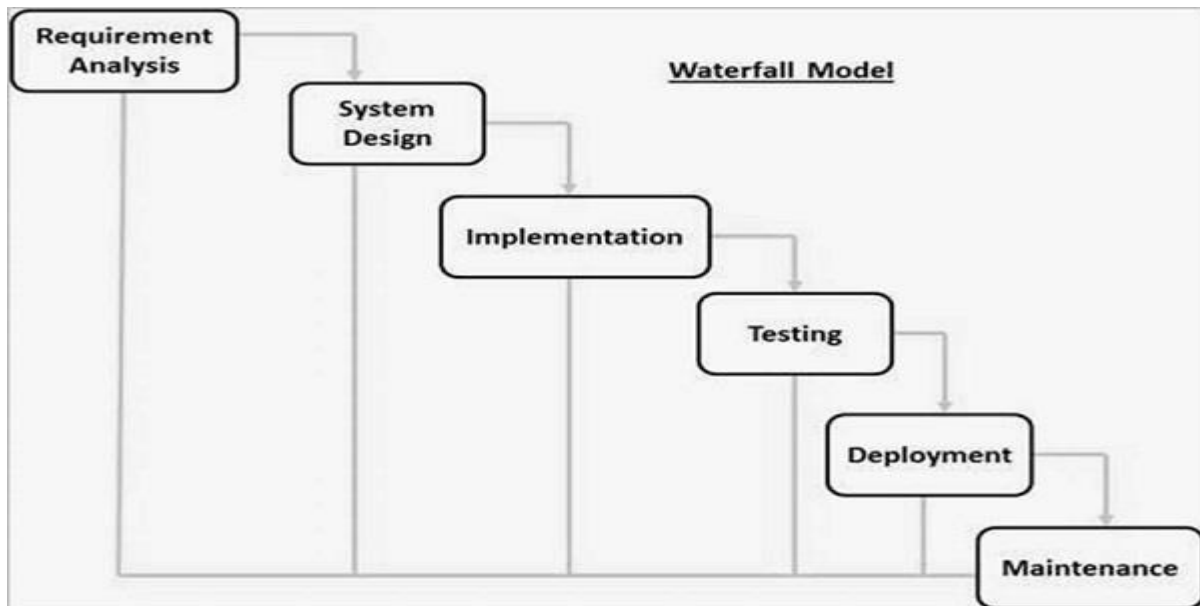
Maintenance is an ongoing stage that involves monitoring, managing, and enhancing the system's performance and functionality. It includes activities such as bug fixes, updates, security patches, and addressing user feedback. Regular maintenance ensures the system remains reliable, secure, and adaptable to changing business needs.

### **System development models**

System development models, also known as software development methodologies are various processes or methods that are chosen for project development depending on the objectives and goals of the project. Many development life cycle models have been developed to achieve various essential objectives. Models specify the various steps of the process and the order in which they are executed.

## 1. Waterfall Model:

Waterfall model is a famous and good version of [SDLC\(System Development Life Cycle\)](#) for software engineering. The waterfall model is a linear and sequential model, which means that a development phase cannot begin until the previous phase is completed. We cannot overlap phases in waterfall model.



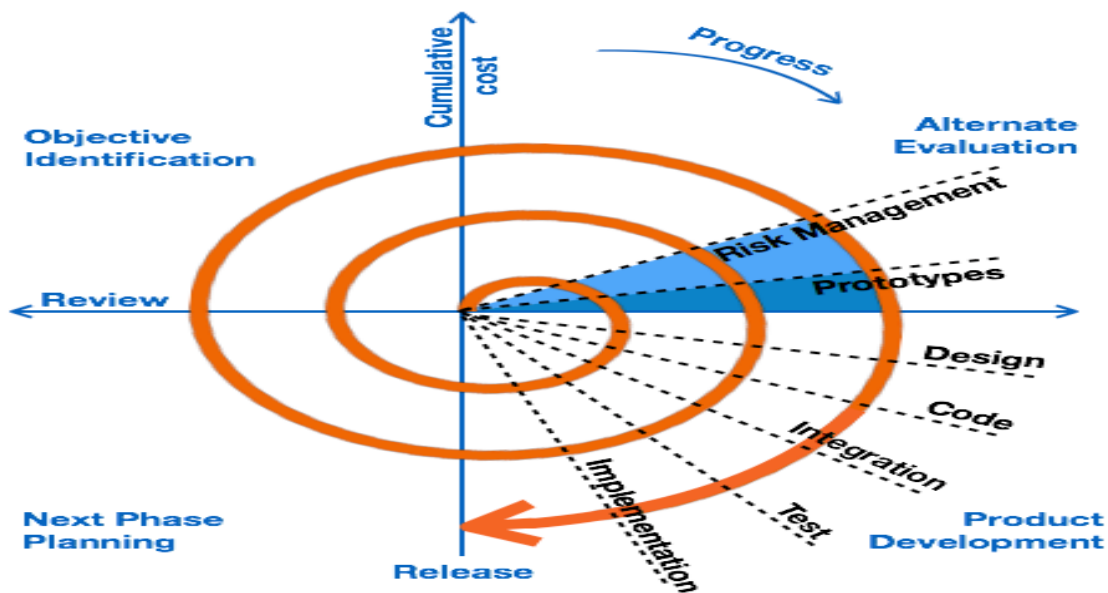
- **Requirement Gathering and analysis** – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- **System Design** – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- **Implementation** – With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
- **Integration and Testing** – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.

- **Deployment of system** – Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** – There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

## Spiral Model:

The Spiral Model is a [Software Development Life Cycle \(SDLC\)](#) model that provides a systematic and iterative approach to software development. In its diagrammatic representation, looks like a spiral with many loops

**The spiral model** can be thought as of a model that has the capability of handling the maximum amount of risks that may occur while developing any software. This model holds the components of almost every other software lifecycle model, i.e. the waterfall models, the prototyping model, the iterative model, the evolutionary model, etc. Due to this, this model is able to handle almost every kind of risk that other models usually fail to handle. However, due to so many components, this model is much more complex than the other software lifecycle models



**The spiral model** overcomes almost every drawback of the other software lifecycle models.

As it can also be seen in the diagram, **the spiral model** is divided into four major quadrants.

### **) First Quadrant**

Sets the objective of the software and analyses all the risks associated with the software.

### **2) Second Quadrant**

This quadrant deals with the complete analysis of each of the risks analyzed in the first quadrant. Apart from that, the risk reduction is also taken care of here.

### **3) Third Quadrant**

This quadrant includes all the development and validation part which includes coding, testing, and other.

### **4) Fourth Quadrant**

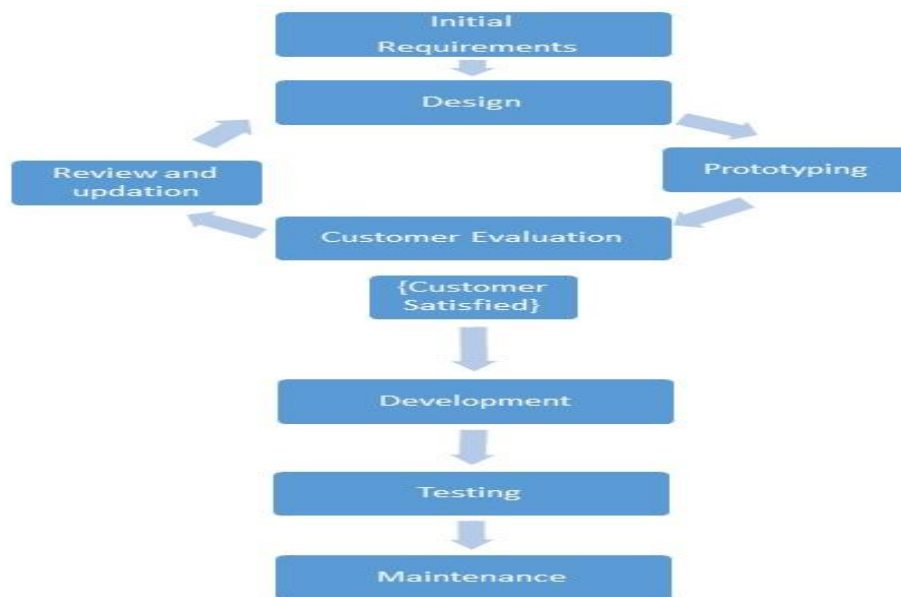
The fourth quadrant deals with the final results that we are getting from the software. It involves the review, planning, and maintenance of the software

## **Prototyping Model:**

The Prototyping Model is one of the most popularly used [Software Development Life Cycle Models \(SDLC models\)](#). This model is used when the customers do not know the exact project requirements beforehand. In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.



A **prototype** means a preliminary model of anything which gives us a rough idea about the basic functionalities that the real model would have



## Why to use prototyping model

While developing software, there are cases wherein the initial stages, we do not know what the overall requirements of the software are. This happens mostly in cases where the customer is not completely sure what he wants his software to look like. So, in these cases, the prototyping model is the best suited because the user can now have some suggestions from the designing team and also make his modifications in the software. Therefore, it is recommended to be used in the following situations

- Customer requirements are not clear and complete.
- The Design is not clear, i.e. how the software must look like.
- The technicians and developers are not clear about the design of the software.

### Introduction to Agile Development

Agile Software Development is a [software development methodology](#).

Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds. These builds are provided in iterations. Each iteration typically lasts from about one to three weeks. It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

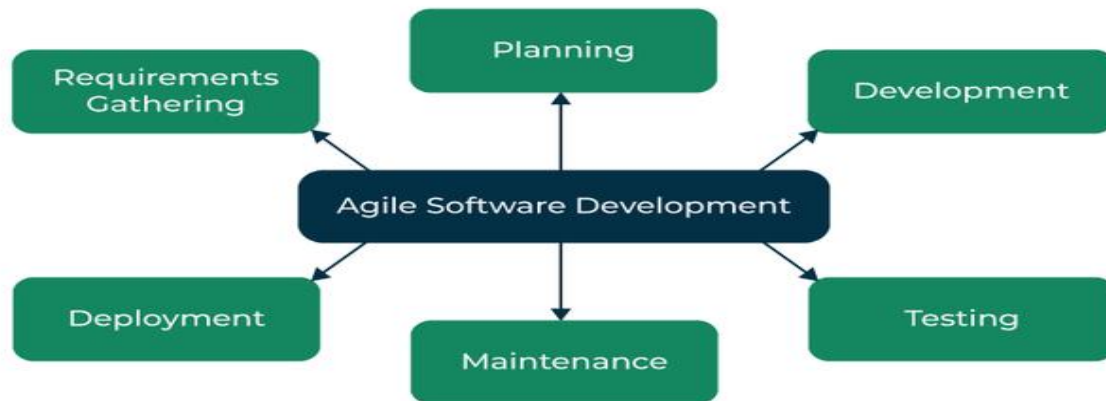
Key principles of Agile Development include:

1. **Iterative and Incremental:** Development is divided into small iterations or increments, allowing for frequent releases and continuous improvement.
2. **Collaboration:** Agile promotes close collaboration between cross-functional teams, including developers, testers, designers, and stakeholders, fostering transparency and shared understanding.
3. **Customer Feedback:** Agile encourages early and frequent feedback from customers or end-users, enabling adjustments to be made quickly based on changing requirements or priorities.
4. **Adaptability:** Agile teams embrace change and respond to it quickly, adjusting plans and priorities as needed to deliver the most valuable features and products.
5. **Continuous Improvement:** Through retrospectives and regular reflection, Agile teams continuously improve their processes and performance

Note:

Popular Agile frameworks include Scrum, Kanban, Extreme Programming (XP), and Lean Software Development. Each framework provides specific practices and guidelines for implementing Agile principles effectively.

## The Agile Software Development Process



1. **Requirements Gathering**: The customer's requirements for the software are gathered and prioritized.
2. **Planning**: The development team creates a plan for delivering the software, including the features that will be delivered in each iteration.
3. **Development**: The development team works to build the software, using frequent and rapid iterations.
4. **Testing**: The software is thoroughly tested to ensure that it meets the customer's requirements and is of high quality.
5. **Deployment**: The software is deployed and put into use.
6. **Maintenance**: The software is maintained to ensure that it continues to meet the customer's needs and expectations

### Advantages Agile Software Development

- Deployment of software is quicker and thus helps in increasing the trust of the customer.
- Can better adapt to rapidly changing requirements and respond faster.
- Helps in getting immediate feedback which can be used to improve the software in the next increment.

- Flexibility and adaptability
- Improved quality and reliability
- Enhanced customer satisfaction
- Faster time-to-market

### **Disadvantages Agile Software Development**

- Agile Development is more code-focused and produces less documentation.
- Face-to-face communication is harder in large-scale organizations
- Maintenance problem

## **Role and attributes of system analyst**

### **Systems Analysis:**

It is a process of collecting and interpreting facts, identifying the problems, and decomposition of a system into its components.

A systems analysis is one method technology professionals can use to identify concerns and suggest improvements to the systems businesses and organizations use in their operations

The system analyst is a person with unique skills – common sense, a structured framework and a disciplined approach to solving problems are a part of the analysis. Therefore the analyst requires a combination of skills, experience, personality and common sense. The fact that a system is designed for a specific user also means that the analyst must have interpersonal skills

### **Roles and responsibilities of a Systems Analyst:**

- Research and evaluate new technology
- Identify the organization need
- Analyze cost and benefits
- Add new functionality to system
- System Design
- System upgradation
- System implementation
- System maintenance

## Qualities/Attributes of a System Analysts:

Knowledge of the organization

Knowledge of computer hardware and software

Problem-solving and critical thinking

Analytical skills

Motivator skills

Good understanding, communication and teaching ability

Problem identification and analysis

### Tools Used by System Analyst:

#### 1. **Modeling Tools:**

- **Unified Modeling Language (UML) Tools:** Software like Enterprise Architect, Visual Paradigm, or Lucidchart are used to create UML diagrams such as use case diagrams, class diagrams, sequence diagrams, etc., to model system requirements and design.
- **Data Modeling Tools:** Tools like ERwin, Toad Data Modeler, or Microsoft Visio help in creating entity-relationship diagrams (ERDs) and data flow diagrams (DFDs) to model data and processes.

Grid Charts:

Grid charts are used to represent the relationship between two sets of factors in a tabular method

#### *System Flow Chart:*

A system flowchart is a pictorial or diagrammatic representation of the logical flow of operations and information of an organization

#### *Decision Tree:*

Some decisions involve a series of steps. The outcome of the first decision guides the second; the third decision depends on the outcome of the second, and so on

#### **Documentation Tools:**

- **Word Processors:** Software like Microsoft Word or Google Docs is used for creating and editing project documentation, including requirements documents, design documents, and user manuals.

## Unit 2. Development Tools

Introduction to DFD and context diagram:

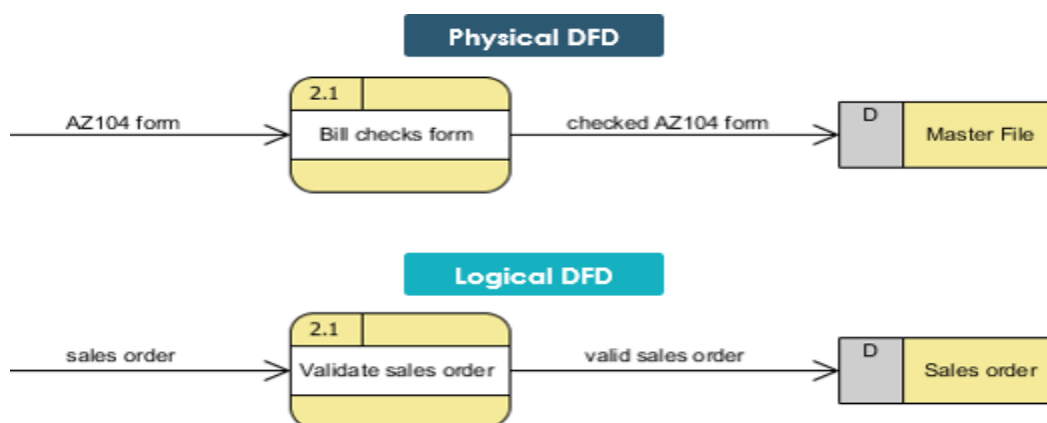
A Data Flow Diagram (DFD) is a graphical representation that illustrates how data flows through a system. It's a tool used in the analysis and design phase of software development to understand the flow of data within a system and between its external entities. DFDs provide a high-level view of the system's functionality and help stakeholders visualize the interactions between different component

### Types of DFD

Data Flow Diagrams are either Logical or Physical.

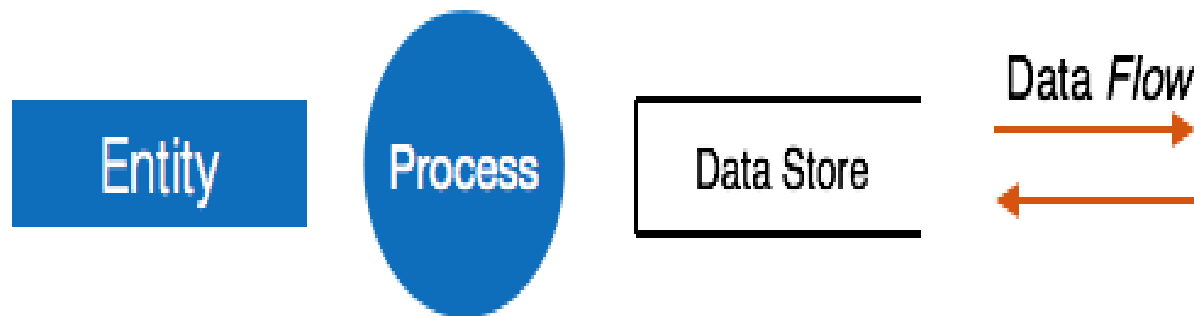
- **Logical DFD** - This type of DFD concentrates on the system process, and flow of data in the system. For example in a Banking software system, how data is moved between different entities.
- **Physical DFD** - This type of DFD shows how the data flow is actually implemented in the system. It is more specific and close to the implementation.
- **Example:**

Physical DFD specifies actual flow of physical documentation, while logical DFD only focus on the information flow in business term.



## DFD Components

DFD can represent Source, destination, storage and flow of data using the following set of components



## DFD rules and tips

- Each process should have at least one input and an output.
- Each data store should have at least one data flow in and one data flow out.
- Data stored in a system must go through a process.
- All processes in a DFD go to another process or a data store
- A single DFD can have a maximum of nine processes and a minimum of three processes.

### **Levels of DFD:**

**0-level DFD:** It represents the entire system as a single bubble and provides an overall picture of the system

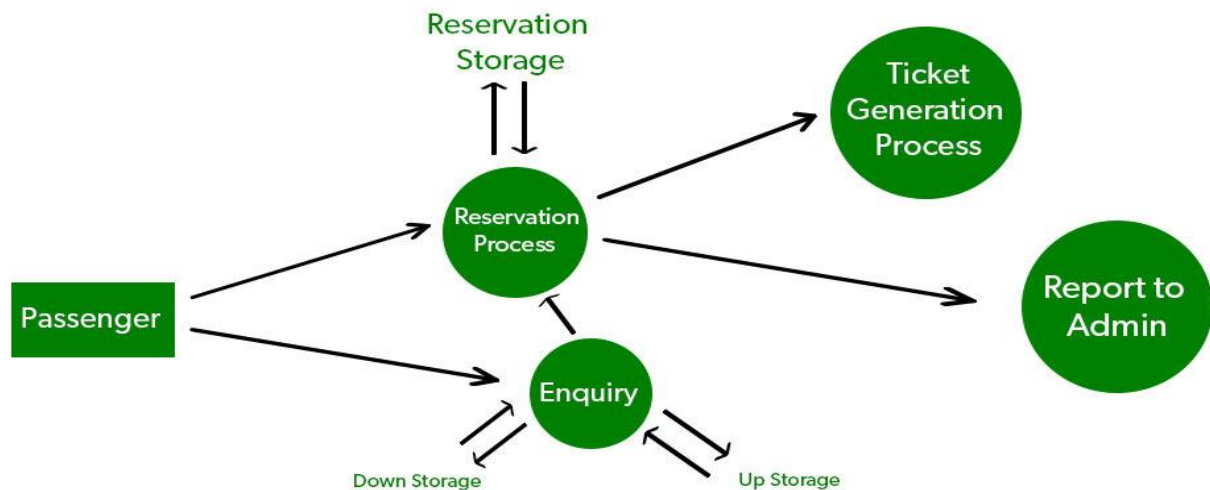
It is also known as a context diagram. It's designed to be an abstraction view, showing the system as a single process with its relationship to external entities. It represents the entire system as a single bubble with input and output data indicated by incoming/outgoing arrows.



### O-LEVEL DFD

- **1-level DFD:** It represents the main functions of the system and how they interact with each other.

In 1-level DFD, the context diagram is decomposed into multiple bubbles/processes. In this level, we highlight the main functions of the system and breakdown the high-level process of 0-level DFD into subprocesses.

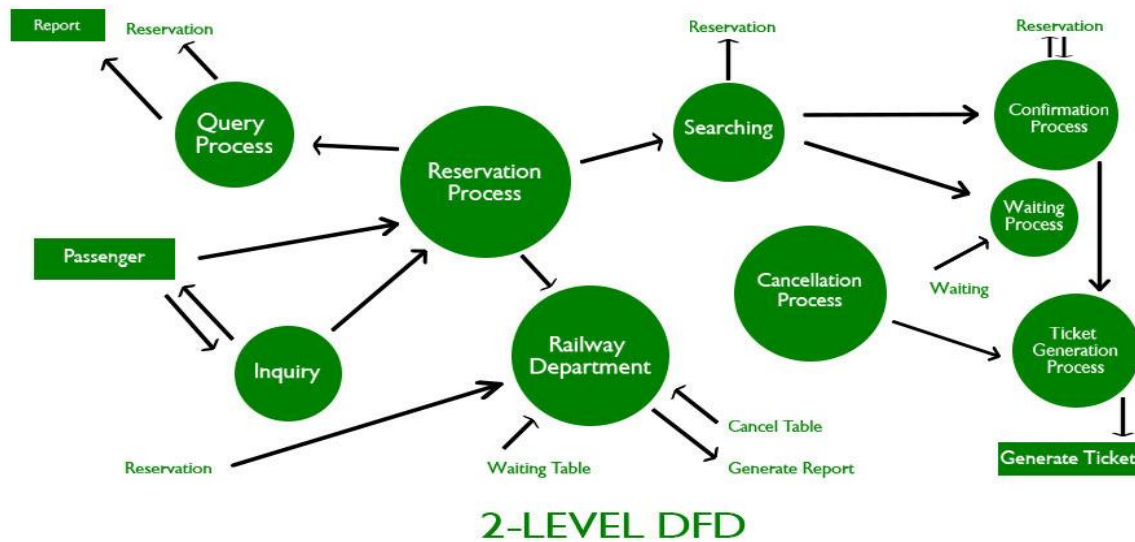


### 1-LEVEL DFD

**2-level DFD:** It represents the processes within each function of the system and how they interact with each other

- 2-level DFD goes one step deeper into parts of 1-level DFD. It can be used to plan or record the specific/necessary detail about the system's functioning.

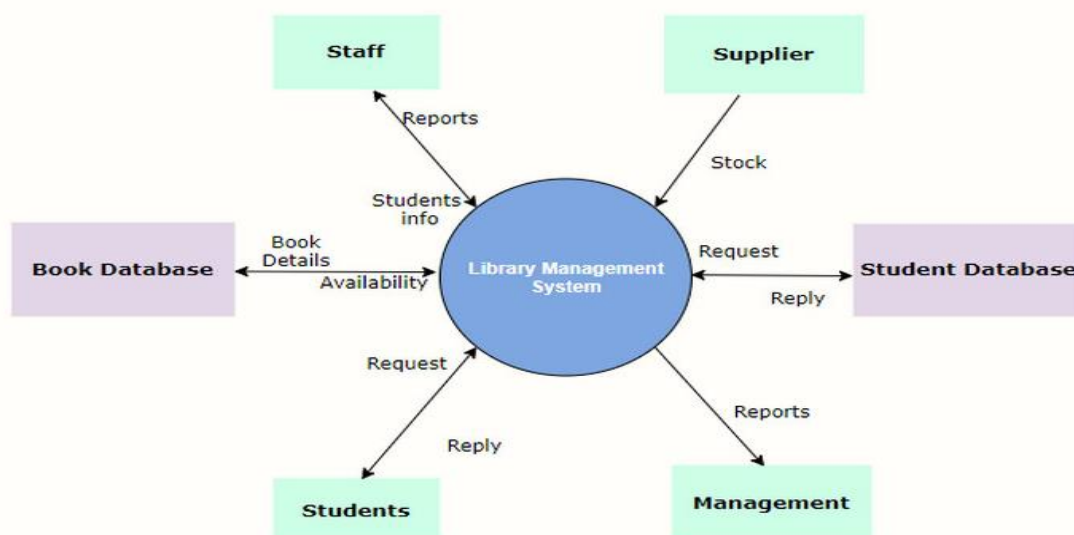




### context diagram:

Context Diagrams are high-level visual representations that show the interactions between a system being developed and its external entities, such as users, other systems, or processes. They provide a big-picture view of how the system fits into its environment without diving into the internal details of the system itself.

**Context diagrams** focus on how external entities interact with your system sometimes called a level 0 data flow diagram.



### **Advantages of DFD:**

It helps us to understand the functioning and the limits of a system.

It is a graphical representation which is very easy to understand as it helps visualize contents.

Data Flow Diagram represent detailed and well explained diagram of system components.

It is used as the part of system documentation file.

Data Flow Diagrams can be understood by both technical or nontechnical person because they are very easy to understand.

### **Disadvantages of DFD:**

- At times DFD can confuse the programmers regarding the system.
- Data Flow Diagram takes long time to be generated, and many times due to this reasons analysts are denied permission to work on it

## **Introduction to ER Diagram:**

ER-modeling is a data modeling method used in software engineering to produce a conceptual data model of an information system. Diagrams created using this ER-modeling method are called Entity-Relationship Diagrams or ER diagrams or ERDs

## **Purpose of ERD**

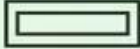
- The database analyst gains a better understanding of the data to be contained in the database through the step of constructing the ERD.
- The ERD serves as a documentation tool.
- Finally, the ERD is used to connect the logical structure of the database to users. In particular, the ERD effectively communicates the logic of the database to users.

### **Symbols Used in ER Model**

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

- **Rectangles:** Rectangles represent Entities in the ER Model.
- **Ellipses:** Ellipses represent Attributes in the ER Model.
- **Diamond:** Diamonds represent Relationships among Entities.
- **Lines:** Lines represent attributes to entities and entity sets with other relationship types.

- **Double Ellipse:** Double Ellipses represent [Multi-Valued Attributes](#).
- **Double Rectangle:** Double Rectangle represents a Weak Entity.

| Figures          | Symbols                                                                           | Represents                                                           |
|------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Rectangle        |  | Entities in ER Model                                                 |
| Ellipse          |  | Attributes in ER Model                                               |
| Diamond          |  | Relationships among Entities                                         |
| Line             |  | Attributes to Entities and Entity Sets with Other Relationship Types |
| Double Ellipse   |  | Multi-Valued Attributes                                              |
| Double Rectangle |  | Weak Entity                                                          |

